```java
package difficult;

import java.util.*;
import java.util.stream.Collectors;

public class ReverseWords {

    public static void main(String[] args) {

        String sentence = "Java streams are powerful";

        String result = Arrays.stream(sentence.split(" "))

                // 🔄 Reverse each word
                .map(word -> new StringBuilder(word).reverse().toString())

                .collect(Collectors.joining(" "));

        System.out.println(result);
    }
}

// avaJ smaerts era lufrewop

//Alternative Way
//
//        .map(word ->
//        Arrays.stream(word.split(""))
//        .reduce("", (rev, ch) -> ch + rev)
//        )

```

```java
package difficult;

import java.util.*;
import java.util.stream.Collectors;

class WorkRecord {
    String name;
    String dept;

    WorkRecord(String name, String dept) {
        this.name = name;
        this.dept = dept;
    }

    public String getName() { return name; }
}

public class Employees3Dept {

    public static void main(String[] args) {

        List<WorkRecord> records = List.of(
                new WorkRecord("John","IT"),
                new WorkRecord("John","Finance"),
                new WorkRecord("John","HR"),
                new WorkRecord("Alice","IT"),
                new WorkRecord("Alice","HR"),
                new WorkRecord("Bob","Finance"),
                new WorkRecord("Bob","Finance"),
                new WorkRecord("David","IT"),
                new WorkRecord("David","Finance"),
                new WorkRecord("David","HR"),
                new WorkRecord("David","Admin")
        );

        Map<String, List<WorkRecord>> grouped = records.stream()
                .collect(Collectors.groupingBy(WorkRecord::getName));

        List<String> result = grouped.entrySet().stream()
                .filter(e -> e.getValue().size() >= 3)
                .map(Map.Entry::getKey)
                .toList();

        System.out.println(result);
    }
}

// [John, David]
```

```java
package difficult;

import java.util.*;
import java.util.stream.Collectors;

enum SalaryRange { LOW, MEDIUM, HIGH }

class Emp {
    String name;
    double salary;

    Emp(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public double getSalary() { return salary; }
}

public class SalaryGrouping {

    public static void main(String[] args) {

        List<Emp> employees = Arrays.asList(
                new Emp("Alice",45000),
                new Emp("Bob",78000),
                new Emp("Charlie",120000),
                new Emp("David",99000),
                new Emp("Eva",30000),
                new Emp("Frank",105000)
        );

        Map<SalaryRange, List<Emp>> result = employees.stream()
                .collect(Collectors.groupingBy(
                        e -> getRange(e.getSalary())
                ));

        System.out.println(result);
    }

    private static SalaryRange getRange(double salary) {
        if (salary < 50000) return SalaryRange.LOW;
        if (salary < 100000) return SalaryRange.MEDIUM;
        return SalaryRange.HIGH;
    }
}

```

```java
package difficult;

import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

public class BigramFrequency {

    public static void main(String[] args) {

        String paragraph = "Java is great and Java is fun Java is powerful";

        String[] words = paragraph.toLowerCase()
                .replaceAll("[^a-z\\s]", "")
                .split(" ");

        List<String> pairs = IntStream.range(0, words.length - 1)

                // ▯ Create pairs
                .mapToObj(i -> words[i] + " " + words[i + 1])

                .toList();

        Map<String, Long> result = pairs.stream()
                .collect(Collectors.groupingBy(
                        Function.identity(),
                        Collectors.counting()
                ));

        System.out.println(result);
    }
}

```

```java
package difficult;

import java.util.*;
import java.util.stream.Collectors;

enum CharType { UPPERCASE, LOWERCASE, DIGIT, OTHER }

public class GroupCharacters {

    public static void main(String[] args) {

        List<Character> chars = Arrays.asList('A','b','3','Z','x','#','7','m','@');

        Map<CharType, List<Character>> result = chars.stream()
                .collect(Collectors.groupingBy(
                        ch -> getType(ch)
                ));

        System.out.println(result);
    }

    private static CharType getType(char c) {
        if (Character.isUpperCase(c)) return CharType.UPPERCASE;
        if (Character.isLowerCase(c)) return CharType.LOWERCASE;
        if (Character.isDigit(c)) return CharType.DIGIT;
        return CharType.OTHER;
    }
}
```

```java
package difficult;

import java.time.LocalDate;
import java.util.*;
import java.util.stream.Collectors;

class Transaction {
    String id;
    LocalDate date;
    double amount;

    Transaction(String id, LocalDate date, double amount) {
        this.id = id;
        this.date = date;
        this.amount = amount;
    }

    public LocalDate getDate() { return date; }
    public double getAmount() { return amount; }
}

public class HighestSpendDay {

    public static void main(String[] args) {

        List<Transaction> list = Arrays.asList(
                new Transaction("T1", LocalDate.of(2024,1,1),300),
                new Transaction("T2", LocalDate.of(2024,1,1),200),
                new Transaction("T3", LocalDate.of(2024,1,2),500),
                new Transaction("T4", LocalDate.of(2024,1,2),300),
                new Transaction("T5", LocalDate.of(2024,1,3),100)
        );

        Map<LocalDate, Double> total = list.stream()

                // 0 Group by date
                .collect(Collectors.groupingBy(
                        Transaction::getDate,
                        Collectors.summingDouble(Transaction::getAmount)
                ));

        Map.Entry<LocalDate, Double> result = total.entrySet().stream()
                .max(Map.Entry.comparingByValue())
                .orElseThrow();

        System.out.println(result);
    }
}
```

```

// 2024-01-02=800.0

```

```java
package difficult;

import java.util.*;

public class LongestWordFinder {

    public static void main(String[] args) {

        String sentence = "The quick, brown fox jumped over the lazy dog!";

        String result = Arrays.stream(
                        sentence.toLowerCase()

                                // 🧹 Remove punctuation
                                .replaceAll("[^a-z\\s]", "")

                                .split(" ")
                )

                // 🏆 Find longest
                .max(Comparator.comparing(String::length))
                .orElse("");

        System.out.println(result);
    }
}
// jumped

```

```java
package difficult;

import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

public class Top3FrequentWords {

    public static void main(String[] args) {

        String paragraph = "Java is great Java is powerful Java is fun";

        Map<String, Long> freq = Arrays.stream(
                        paragraph.toLowerCase()
                                .replaceAll("[^a-z\\s]", "")
                                .split(" ")
                )
                .collect(Collectors.groupingBy(
                        Function.identity(),
                        Collectors.counting()
                ));

        List<Map.Entry<String, Long>> result = freq.entrySet().stream()

                // ⬇ Sort descending
                .sorted(Map.Entry.<String, Long>comparingByValue().reversed())

                // ✂ Top 3
                .limit(3)

                .toList();

        System.out.println(result);
    }
}

// [java=3, is=3, great=1]

// Get top frequencies only (Alternative Way)

//List<Long> topFreq = freq.values().stream()
//        .distinct()
//        .sorted(Comparator.reverseOrder())
//        .limit(3)
//        .toList();

```

```java
package difficult;

import java.util.*;
import java.util.stream.IntStream;

public class SlidingWindowAverage {

    public static void main(String[] args) {

        List<Integer> input = Arrays.asList(5,10,15,20,25,30);
        int window = 3;

        List<Double> result = IntStream.range(0, input.size() - window + 1)

                // 🔹 Create sublists
                .mapToObj(i -> input.subList(i, i + window))

                // 🔹 Compute avg for each window
                .map(list -> list.stream()
                        .mapToInt(Integer::intValue)
                        .average()
                        .orElse(0.0)
                )

                .toList();

        System.out.println(result);
    }
}

// [10.0, 15.0, 20.0, 25.0]

```

```java
package difficult;

import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

class Employee {
    String name;
    String department;
    double salary;

    Employee(String name, String department, double salary) {
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    public String getName() { return name; }
}

public class MostCommonFirstLetter {

    public static void main(String[] args) {

        List<Employee> employees = List.of(
                new Employee("John","IT",60000),
                new Employee("Alice","HR",55000),
                new Employee("Bob","IT",75000),
                new Employee("Carol","HR",70000),
                new Employee("David","Finance",65000),
                new Employee("Eve","IT",80000),
                new Employee("Frank","Finance",64000),
                new Employee("Ala","Operations",44000)
        );

        Map<Character, Long> freq = employees.stream()

                // ▯ Extract first character
                .map(e -> e.getName().charAt(0))

                // ▯ Count frequency
                .collect(Collectors.groupingBy(
                        Function.identity(),
                        Collectors.counting()
                ));

        Map.Entry<Character, Long> result = freq.entrySet().stream()
                .max(Map.Entry.comparingByValue())
```

```
                .orElseThrow();

        System.out.println(result);
    }
}

// A=2

```